

Name : Venna Meenakshi Reddy

Register Number : 192471048

Course Code : CSA0608

Course Name : Design and Analysis of Algorithms

Faculty : Dr. P. Sriramya

Algorithm Analysis Fundamentals

Time complexity analysis of iterative and recursive algorithms; verify Big-O, Ω , Θ for sample programs

Experiment 1: Linear Search (Iterative)

- **Objective:** To analyze time complexity of linear search.
- **Problem Statement:** Write a Python program to search for a key in an array using linear search.
- **Sample Input:** [10, 25, 30, 45, 50], Key = 30
- **Sample Output:** Key found at index 2
- **Complexity:** Ω : $O(1)$, O : $O(n)$, Θ : $\Theta(n)$
- **Explanation:** Demonstrates sequential scanning and inefficiency for large inputs.

Experiment 2: Binary Search (Recursive)

- **Objective:** To compare recursive binary search complexity.
- **Problem Statement:** Implement recursive binary search in Python.
- **Sample Input:** [5, 10, 15, 20, 25], Key = 20
- **Sample Output:** Key found at index 3
- **Complexity:** Ω : $O(1)$, O : $O(\log_{10} n)$, Θ : $\Theta(\log_{10} n)$
- **Explanation:** Shows divide-and-conquer and logarithmic efficiency.

Experiment 3: Factorial (Iterative vs Recursive)

- **Objective:** To compare iterative and recursive factorial computation.
- **Problem Statement:** Write programs to compute factorial using both methods.
- **Sample Input:** $n = 5$
- **Sample Output:** 120
- **Complexity:** Both $O(n)$
- **Explanation:** Highlights recursion depth vs iterative loops.

Experiment 4: Fibonacci Series

- **Objective:** To analyze recursive vs iterative Fibonacci generation.
- **Problem Statement:** Generate first n Fibonacci numbers.
- **Sample Input:** $n = 6$
- **Sample Output:** [0, 1, 1, 2, 3, 5]
- **Complexity:** Iterative $O(n)$, Recursive naïve $O(2n)$, Recursive with memoization $O(n)$
- **Explanation:** Illustrates exponential recursion vs efficient iteration.

Experiment 5: Matrix Multiplication

- **Objective:** To analyze nested loop complexity.
- **Problem Statement:** Multiply two matrices using loops.
- **Sample Input:** $A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$, $B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$
- **Sample Output:** $\begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$
- **Complexity:** $O(n^3)$
- **Explanation:** Shows cubic growth due to nested loops.

Experiment 6: Merge Sort

- **Objective:** To analyze divide-and-conquer sorting.
- **Problem Statement:** Implement merge sort.
- **Sample Input:** [38, 27, 43, 3, 9, 82, 10]
- **Sample Output:** [3, 9, 10, 27, 38, 43, 82]
- **Complexity:** Always $O(n \log n)$
- **Explanation:** Demonstrates guaranteed efficiency of divide-and-conquer.

Experiment 7: Quick Sort

- **Objective:** To analyze recursive quick sort.
- **Problem Statement:** Implement quick sort.
- **Sample Input:** [10, 7, 8, 9, 1, 5]
- **Sample Output:** [1, 5, 7, 8, 9, 10]
- **Complexity:** Ω : $O(n \log_{10} n)$, O : $O(n^2)$, Θ : $\Theta(n \log_{10} n)$
- **Explanation:** Shows how pivot choice affects performance.

Experiment 8: Tower of Hanoi

- **Objective:** To analyze exponential recursion.
- **Problem Statement:** Solve Tower of Hanoi for n disks.
- **Sample Input:** n = 3
- **Sample Output:** Sequence of moves.
- **Complexity:** $O(2^n)$
- **Explanation:** Illustrates exponential growth in recursive calls.

Experiment 9: GCD (Euclidean Algorithm)

- **Objective:** To analyze recursive GCD.
- **Problem Statement:** Implement Euclidean algorithm.
- **Sample Input:** a=48, b=18
- **Sample Output:** 6
- **Complexity:** $O(\log_{10} n)$
- **Explanation:** Shows efficient recursion with logarithmic complexity.

Experiment 10: Insertion Sort

- **Objective:** To analyze iterative sorting.
- **Problem Statement:** Implement insertion sort.
- **Sample Input:** [12, 11, 13, 5, 6]
- **Sample Output:** [5, 6, 11, 12, 13]
- **Complexity:** Ω : $O(n)$, O : $O(n^2)$, Θ : $\Theta(n^2)$
- **Explanation:** Demonstrates input sensitivity (sorted vs unsorted).

Experiment 11: Heap Sort

- **Objective:** To analyze heap-based sorting.
- **Problem Statement:** Implement heap sort.
- **Sample Input:** [4, 10, 3, 5, 1]
- **Sample Output:** [1, 3, 4, 5, 10]
- **Complexity:** Always $O(n \log n)$
- **Explanation:** Consistent performance using heap structure.

Experiment 12: Counting Inversions

- **Objective:** To analyze modified merge sort.
- **Problem Statement:** Count inversions in an array.
- **Sample Input:** [2, 4, 1, 3, 5]
- **Sample Output:** 3
- **Complexity:** $O(n \log n)$
- **Explanation:** Shows divide-and-conquer applied beyond sorting.

Experiment 13: Maximum Subarray Sum

- **Objective:** To compare Kadane's vs divide-and-conquer.
- **Problem Statement:** Find maximum subarray sum.
- **Sample Input:** [-2, -3, 4, -1, -2, 1, 5, -3]
- **Sample Output:** 7
- **Complexity:** Kadane's $O(n)$, Divide & Conquer $O(n \log n)$
- **Explanation:** Highlights efficiency differences between approaches.

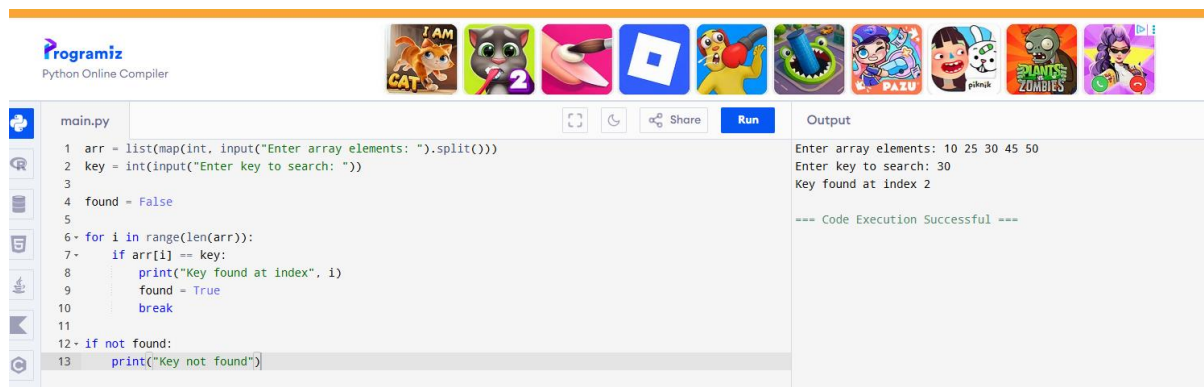
Experiment 14: Exponentiation

- **Objective:** To compare iterative vs recursive fast power.
- **Problem Statement:** Compute x^n .
- **Sample Input:** $x=2, n=10$
- **Sample Output:** 1024
- **Complexity:** Iterative $O(n)$, Recursive fast power $O(\log n)$
- **Explanation:** Shows how recursion reduces complexity.

Experiment 15: Closest Pair of Points

- **Objective:** To analyze brute force vs divide-and-conquer.
- **Problem Statement:** Find closest pair of points in 2D.
- **Sample Input:** $[(1, 2), (4, 5), (7, 8), (3, 1)]$
- **Sample Output:** Closest pair $(1, 2) - (3, 1)$, Distance = $\sqrt{5}$
- **Complexity:** Brute force $O(n^2)$, Optimized $O(n \log n)$
- **Explanation:** Demonstrates quadratic complexity and motivates optimization.

Experiment 1: Linear Search (Iterative)

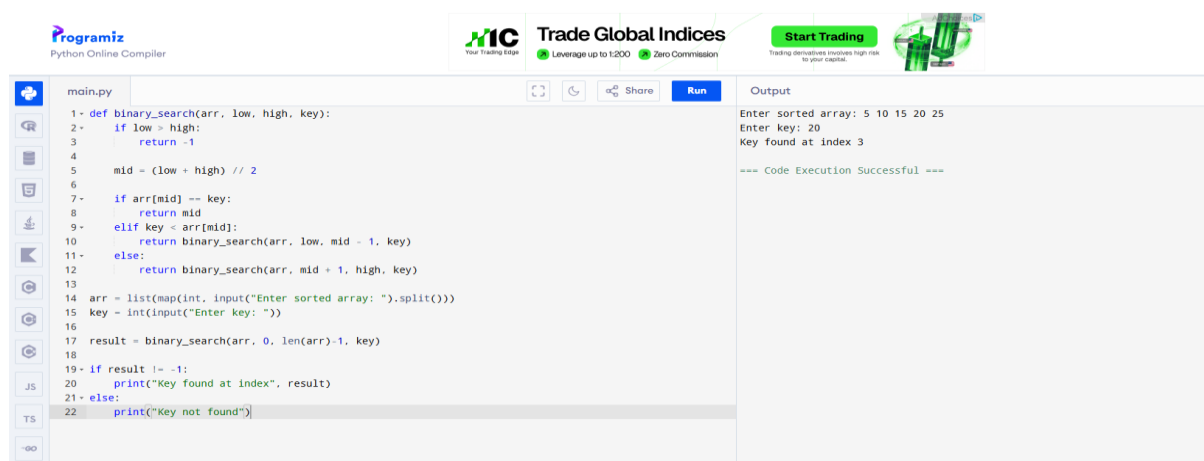


The screenshot shows the Programiz Python Online Compiler interface. The code in `main.py` implements a linear search algorithm:

```
1 arr = list(map(int, input("Enter array elements: ").split()))
2 key = int(input("Enter key to search: "))
3
4 found = False
5
6 for i in range(len(arr)):
7     if arr[i] == key:
8         print("Key found at index", i)
9         found = True
10        break
11
12 if not found:
13     print("Key not found")
```

The output shows the user inputting the array elements "10 25 30 45 50" and the key "30". The program outputs "Key found at index 2" and "=== Code Execution Successful ===".

Experiment 2: Binary Search (Recursive)



The screenshot shows the Programiz Python Online Compiler interface. The code in `main.py` implements a recursive binary search algorithm:

```
1 def binary_search(arr, low, high, key):
2     if low > high:
3         return -1
4
5     mid = (low + high) // 2
6
7     if arr[mid] == key:
8         return mid
9     elif key < arr[mid]:
10        return binary_search(arr, low, mid - 1, key)
11    else:
12        return binary_search(arr, mid + 1, high, key)
13
14 arr = list(map(int, input("Enter sorted array: ").split()))
15 key = int(input("Enter key: "))
16
17 result = binary_search(arr, 0, len(arr)-1, key)
18
19 if result != -1:
20     print("Key found at index", result)
21 else:
22     print("Key not found")
```

The output shows the user inputting the sorted array elements "5 10 15 20 25" and the key "20". The program outputs "Key found at index 3" and "=== Code Execution Successful ===".

Experiment 3: Factorial (Iterative vs Recursive)


Python Online Compiler


Your Trading Edge

Trade Global Indices
Leverage up to 1:200
Zero Commission

Start Trading
Trading derivatives involves high risk to your capital.



main.py

```
1- def recursive_factorial(n):
2-     if n == 0 or n == 1:
3-         return 1
4-     return n * recursive_factorial(n - 1)
5
6 n = int(input("Enter a number: "))
7
8 fact = 1
9- for i in range(1, n + 1):
10-     fact *= i
11
12 print("Iterative:", fact)
13 print("Recursive:", recursive_factorial(n))
```

Output

Enter a number: 5
Iterative: 120
Recursive: 120

=== Code Execution Successful ===

Experiment 4: Fibonacci Series


Python Online Compiler


Your Trading Edge

Trade Global Indices
Leverage up to 1:200
Zero Commission

Start Trading
Trading derivatives involves high risk to your capital.



main.py

```
1- def fibonacci(n):
2-     if n == 0:
3-         return 0
4-     if n == 1:
5-         return 1
6-     return fibonacci(n - 1) + fibonacci(n - 2)
7
8 n = int(input("Enter number of terms: "))
9
10 print("Iterative:")
11 a, b = 0, 1
12
13- for i in range(n):
14-     if i == 0:
15-         print(a, end=" ")
16-     elif i == 1:
17-         print(b, end=" ")
18-     else:
19-         c = a + b
20-         print(c, end=" ")
21-         a, b = b, c
22
23 print()
24
25 print("Recursive:")
26- for i in range(n):
27-     print(fibonacci(i), end=" ")
```

Output

Enter number of terms: 6
Iterative:
0 1 1 2 3 5
Recursive:
0 1 1 2 3 5

=== Code Execution Successful ===

Experiment 5: Matrix Multiplication


Python Online Compiler


Your Trading Edge

Trade Global Indices
Leverage up to 1:200 Zero Commission

Start Trading
Trading derivatives involves high risk to your capital.



main.py

```
1 A = []
2 B = []
3
4 print("Enter first 2x2 matrix:")
5 for i in range(2):
6     row = list(map(int, input().split()))
7     A.append(row)
8
9 print("Enter second 2x2 matrix:")
10 for i in range(2):
11     row = list(map(int, input().split()))
12     B.append(row)
13
14 result = [[0, 0], [0, 0]]
15
16 for i in range(2):
17     for j in range(2):
18         for k in range(2):
19             result[i][j] += A[i][k] * B[k][j]
20
21 print("Result:")
22 for row in result:
23     print(row)
```

Output

Enter first 2x2 matrix:
1 2
3 4

Enter second 2x2 matrix:
5 6
7 8

Result:
[19, 22]
[43, 50]

=== Code Execution Successful ===

Experiment 6: Merge Sort


Python Online Compiler


Your Trading Edge

Trade Global Indices
Leverage up to 1:200 Zero Commission

Start Trading
Trading derivatives involves high risk to your capital.



main.py

```
1 def merge_sort(arr):
2     if len(arr) > 1:
3         mid = len(arr) // 2
4         left = arr[:mid]
5         right = arr[mid:]
6
7         merge_sort(left)
8         merge_sort(right)
9
10        i = j = k = 0
11
12        while i < len(left) and j < len(right):
13            if left[i] < right[j]:
14                arr[k] = left[i]
15                i += 1
16            else:
17                arr[k] = right[j]
18                j += 1
19                k += 1
20
21        while i < len(left):
22            arr[k] = left[i]
23            i += 1
24            k += 1
25
26        while j < len(right):
27            arr[k] = right[j]
28            j += 1
29            k += 1
30
31 arr = list(map(int, input("Enter array elements: ").split()))
32
33 merge_sort(arr)
34
35 print("Sorted array:", arr)
```

Output

Enter array elements: 38 27 43 3 9 82 10
Sorted array: [3, 9, 10, 27, 38, 43, 82]

=== Code Execution Successful ===

Experiment 7: Quick Sort


Python Online Compiler


Your Trading Edge

Trade Global Indices
Leverage up to 1200
Zero Commission

Start Trading
Trading derivatives involves high risk to your capital.



```
main.py
1- def quick_sort(arr):
2-     if len(arr) <= 1:
3-         return arr
4-
5-     pivot = arr[0]
6-     left = []
7-     right = []
8-
9-     for i in arr[1:]:
10-         if i <= pivot:
11-             left.append(i)
12-         else:
13-             right.append(i)
14-
15-     return quick_sort(left) + [pivot] + quick_sort(right)
16-
17- arr = list(map(int, input("Enter array elements: ").split()))
18-
19- sorted_arr = quick_sort(arr)
20-
21- print("Sorted array:", sorted_arr)
```

Output
Enter array elements: 10 7 8 9 1 5
Sorted array: [1, 5, 7, 8, 9, 10]

=== Code Execution Successful ===

Experiment 8: Tower of Hanoi


Python Online Compiler


Build, brand, and back your business with GoDaddy.



```
main.py
1- def tower(n, source, auxiliary, destination):
2-     if n == 1:
3-         print("Move disk 1 from", source, "to", destination)
4-         return
5-
6-     tower(n - 1, source, destination, auxiliary)
7-     print("Move disk", n, "from", source, "to", destination)
8-     tower(n - 1, auxiliary, source, destination)
9-
10- n = int(input("Enter number of disks: "))
11-
12- tower(n, 'A', 'B', 'C')
```

Output
Enter number of disks: 3
Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B
Move disk 3 from A to C
Move disk 1 from B to A
Move disk 2 from B to C
Move disk 1 from A to C

=== Code Execution Successful ===

Experiment 9: GCD (Euclidean Algorithm)


Python Online Compiler



```
main.py
1- def gcd(a, b):
2-     if b == 0:
3-         return a
4-     return gcd(b, a % b)
5-
6- a = int(input("Enter first number: "))
7- b = int(input("Enter second number: "))
8-
9- print("GCD =", gcd(a, b))
```

Output
Enter first number: 48
Enter second number: 18
GCD = 6

=== Code Execution Successful ===

Experiment 10: Insertion Sort


Python Online Compiler

 Now hiring for *accounts receivable* roles [Apply today](#)

 main.py    [Run](#)

```
1 arr = list(map(int, input("Enter array elements: ").split()))
2
3 for i in range(1, len(arr)):
4     key = arr[i]
5     j = i - 1
6
7     while j >= 0 and arr[j] > key:
8         arr[j + 1] = arr[j]
9         j -= 1
10
11     arr[j + 1] = key
12
13 print("Sorted array:", arr)
```

Output

```
Enter array elements: 12 11 13 5 6
Sorted array: [5, 6, 11, 12, 13]

=== Code Execution Successful ===
```

Experiment 11: Heap Sort


Python Online Compiler

 Build, brand, and back your business with GoDaddy.

 Ricardo 9LIVES

 main.py    [Run](#)

```
1 def heapify(arr, n, i):
2     largest = i
3     left = 2 * i + 1
4     right = 2 * i + 2
5
6     if left < n and arr[left] > arr[largest]:
7         largest = left
8
9     if right < n and arr[right] > arr[largest]:
10        largest = right
11
12    if largest != i:
13        arr[i], arr[largest] = arr[largest], arr[i]
14        heapify(arr, n, largest)
15
16 def heap_sort(arr):
17     n = len(arr)
18
19     for i in range(n // 2 - 1, -1, -1):
20         heapify(arr, n, i)
21
22     for i in range(n - 1, 0, -1):
23         arr[0], arr[i] = arr[i], arr[0]
24         heapify(arr, i, 0)
25
26 arr = list(map(int, input("Enter array elements: ").split()))
27
28 heap_sort(arr)
29
30 print("Sorted array:", arr)
```

Output

```
Enter array elements: 4 10 3 5 1
Sorted array: [1, 3, 4, 5, 10]

=== Code Execution Successful ===
```

Experiment 12: Counting Inversions



Python Online Compiler



Build, brand, and back your business with GoDaddy.



UNIVERSITY OF CALicut, MALAPPURAM, INDIA



Ricardo de To
7LIVES.ES

main.py






```

1- def count_inversions(arr):
2-     count = 0
3-
4-     for i in range(len(arr)):
5-         for j in range(i + 1, len(arr)):
6-             if arr[i] > arr[j]:
7-                 count += 1
8-
9-     return count
10
11 arr = list(map(int, input("Enter array elements: ").split()))
12
13 print("Number of inversions:", count_inversions(arr))

```

Output

```

Enter array elements: 2 4 1 3 5
Number of inversions: 3

=== Code Execution Successful ===

```

Experiment 13: Maximum Subarray Sum

Programiz
Python Online Compiler

Build, brand, and back your business with GoDaddy.

Ricardo de Tomás
FLORES

main.py
🔍 🔒 🔄 Share
Run
Output

```

1 arr = list(map(int, input("Enter array elements: ").split()))
2
3 max_current = arr[0]
4 max_global = arr[0]
5
6- for i in range(1, len(arr)):
7     max_current = max(arr[i], max_current + arr[i])
8     max_global = max(max_global, max_current)
9
10 print("Maximum Subarray Sum:", max_global)
```

Enter array elements: -2 -3 4 -1 -2 1 5 -3
Maximum Subarray Sum: 7

=== Code Execution Successful ===

Experiment 14: Exponentiation





Build, brand, and back your business with GoDaddy.
 

main.py






```

1- def power_recursive(x, n):
2-     if n == 0:
3-         return 1
4-     if n % 2 == 0:
5-         half = power_recursive(x, n // 2)
6-         return half * half
7-     else:
8-         return x * power_recursive(x, n - 1)
9
10 x = int(input("Enter base: "))
11 n = int(input("Enter exponent: "))
12
13 # Iterative
14 result = 1
15 for i in range(n):
16     result *= x
17
18 print("Iterative Power:", result)
19 print("Recursive Power:", power_recursive(x, n))

```

Output

Enter base: 2
 Enter exponent: 10
 Iterative Power: 1024
 Recursive Power: 1024

 === Code Execution Successful ===

Experiment 15: Closest Pair of Points

main.py

```
1 import math
2
3 n = int(input("Enter number of points: "))
4
5 points = []
6
7- for i in range(n):
8     x, y = map(int, input("Enter x y: ").split())
9     points.append((x, y))
10
11 min_dist = float('inf')
12 pair = ()
13
14- for i in range(n):
15-     for j in range(i + 1, n):
16-         d = math.sqrt((points[i][0] - points[j][0])**2 +
17-                        (points[i][1] - points[j][1])**2)
18-
19-         if d < min_dist:
20-             min_dist = d
21-             pair = (points[i], points[j])
22
23 print("Closest Pair:", pair)
24 print("Distance:", min_dist)
```

Output

Enter number of points: 4
Enter x y: 1 2
Enter x y: 4 5
Enter x y: 7 8
Enter x y: 3 1
Closest Pair: ((1, 2), (3, 1))
Distance: 2.23606797749979

=== Code Execution Successful ===